

Resolución Técnica – TP Bomba Binaria

Este documento presenta una descripción detallada y paso a paso del proceso seguido para resolver las diferentes fases del trabajo práctico denominado “Bomba Binaria”. El enfoque se basó en analizar cuidadosamente el código ensamblador, utilizar el debugger GDB para inspeccionar la memoria y realizar scripts en Python que automatizaran tareas específicas y complejas.

Fase 1 – Comparación de Cadenas

Análisis del código ensamblador

Al inicio, revisé la función `phase_1` y observé que la misma comparaba la cadena que yo ingresaba con otra almacenada dentro del binario. El código ensamblador indicaba claramente una comparación directa:

```
401def: lea rsi, [rip+0xc7c62]      ; carga puntero a string constante
401df6: call 4022b9 <strings_not_equal>
401dfb: test eax, eax
401dfd: jne 401e04 <explode_bomb>
```

Resolución utilizando GDB

Para obtener el valor correcto:

- Coloqué un breakpoint en `explode_bomb` y otro justo después de la comparación.
- Calculé la dirección del string sumando `rip` y el offset dado.
- Inspeccioné esta dirección con el comando de GDB:

```
x/s 0x4c9a58
```

Encontré así la cadena requerida:

```
"Confía en el tiempo, que suele dar dulces salidas a muchas amargas dificultades"
```

Ingresar este valor resolvió exitosamente la primera fase.

Fase 2 – Validación Numérica con Condiciones a Nivel de Bits

Análisis del código ensamblador

La función `phase_2` recibe como entrada una línea con dos números enteros. El flujo principal comienza con el parseo de ambos valores utilizando `strtoul`:

```
401e77: call 406860 <__strtoul>          ; primer número
401e8f: call 406860 <__strtoul>          ; segundo número
```

El primer número se almacena en `rbx`, y el segundo en `eax`, luego copiado a `edx`. Estos valores se usan para calcular el primer argumento de la función auxiliar `misterio`, que es:

```
401e96: lea edi, [rbx+rax*1-0x20]
```

Es decir, se calcula:

$$a = x + y - 32$$

La llamada resultante a `misterio` es:

```
misterio(x + y - 32, x, y)
```

Lógica interna de `misterio`

La función `misterio` impone dos restricciones principales que deben cumplirse para evitar que la bomba explote.

1. Conteo de bits en 1

```
401e23: loop de 32 iteraciones desplazando a a la derecha
401e25: sar eax, cl           ; shift
401e27: and eax, 0x1         ; extrae bit menos significativo
401e2a: add edx, eax         ; acumula si era 1
...
401e34: cmp edx, 0x0b        ; compara contra 11
401e37: jne explode_bomb
```

Se recorre bit a bit el valor de $a = x + y - 32$ y se contabilizan cuántos bits están en 1. La primera condición es que el total debe ser exactamente 11.

2. XOR con signo negativo

```
401e39: xor ebx, ebp         ; x ^ y
401e3b: js 401e42            ; si es negativo, continúa
401e3d: call explode_bomb    ; si no, falla
```

Luego se evalúa si $x \wedge y$ tiene el bit de signo activado, es decir, si es un número negativo en representación con signo (complemento a dos).

Condiciones derivadas

Del análisis anterior, se infiere que los valores `x` e `y` deben cumplir simultáneamente:

- Que $x + y - 32$ tenga exactamente **11 bits en 1**.
- Que $x \wedge y$ sea **negativo**.

Estrategia de resolución

Para automatizar la búsqueda de pares válidos, se escribió el siguiente script en Python:

```
for x in range(-10000, 10000):
    for y in range(-10000, 10000):
        a = x + y - 32
        if (bin(a).count('1') == 11) and ((x ^ y) < 0):
            print(x, y)
```

Este script recorre un rango amplio y evalúa las dos condiciones requeridas. Uno de los pares válidos resultantes fue:

-7921 10000

Al ingresar estos valores, la función `misterio` retornó exitosamente sin activar `explode_bomb`, desactivando así la segunda fase correctamente.

Fase 3 – Búsqueda Binaria Recursiva con Suma de Caracteres

Análisis ensamblador completo

La función `phase_3` solicita una entrada de dos valores, parseados con:

```
402064: lea rsi, [rip+0xc502e] ; "%s %d"
402073: call 4074d0 <__isoc99_sscanf>
```

Esto indica que el programa espera primero un **string** y luego un **número entero**. La palabra es almacenada en memoria dinámica (`malloc` en `0x402056`) y el número se guarda temporalmente en la pila (`[rsp]`).

Posteriormente, se invoca la función `readlines`, que carga el contenido del archivo `"palabras.txt"`:

```
401ed7: lea rdi, [rip+0xc51ae] ; dirección a "palabras.txt"
401ede: call 410aa0 <_IO_new_fopen>
```

Este archivo contiene **10.784 palabras ordenadas alfabéticamente**, que se cargan en un arreglo de punteros.

Lógica de la función cuenta

La función `cuenta(string, arreglo, ini, fin)` implementa una **búsqueda binaria recursiva**. En cada paso:

- Se compara la palabra con el centro del intervalo.
- Se suma el valor ASCII del primer carácter de cada nodo visitado.
- Si la palabra no se encuentra, la bomba explota.

La explosión ocurre en:

402022: `call explode_bomb`

El valor total retornado por `cuenta` es usado como validación.

Validaciones posteriores

El valor de `cuenta` debe cumplir dos condiciones adicionales:

1. Estar en el rango [401, 799]:

```
4020a7: lea eax,[eax-0x191]
4020ad: cmp eax,0x18e
4020b2: ja explode_bomb
```

2. Coincidir con el número ingresado:

```
4020b4: cmp [rsp],ebx
4020b7: jne explode_bomb
```

Simulación exacta en Python

Se replicó la función `cuenta` con el siguiente script:

```
def cuenta(palabra, arreglo, ini, fin):
    if ini > fin:
        raise Exception("explota la bomba")
    medio = (ini + fin) // 2
    c = ord(arreglo[medio][0])
    if palabra == arreglo[medio]:
        return c
    elif palabra < arreglo[medio]:
        return c + cuenta(palabra, arreglo, ini, medio - 1)
    else:
        return c + cuenta(palabra, arreglo, medio + 1, fin)
```

Se aplicó a las 10.784 palabras para encontrar todas las que produjeran valores válidos.

Palabras válidas encontradas

Entre las palabras con resultados en el rango [401, 799], se encontraron:

- `abatatar` → 782
- `aboquillar` → 685

- abstenerse → 782
- zumear → 779
- zurzar → 739

Para resolver esta fase se utilizó:

`abatatar 782`

Ya que "abatatar" es válida y retorna exactamente el valor permitido. La fase fue superada exitosamente sin que la bomba explotara.

Fase 4 – Validación por Indexación y Suma de Caracteres

Análisis del código ensamblador

La función `phase_4` verifica que el input del usuario sea una cadena de **exactamente seis caracteres**, y luego aplica una lógica de suma indexada utilizando una tabla de enteros.

1. Verificación de longitud La longitud del string se verifica con:

```
402128: call 402298 <string_length>
40212d: cmp eax, 0x6
402130: jne explode_bomb
```

Cualquier input cuya longitud no sea 6 activa la bomba.

2. Inicialización y carga del arreglo Luego, se define el puntero al arreglo de constantes en memoria (`array.0`), de la siguiente forma:

```
40213e: lea rsi, [rip+0xcbcfb] ; dirección → array.0
```

Este arreglo se accede con índices de 4 bits, derivados de cada carácter del string.

3. Suma de valores indexados La suma se realiza iterando sobre los seis caracteres del string. Para cada uno:

- Se extraen los 4 bits bajos del carácter (`ord(c) & 0x0F`)
- Se usa ese valor como índice en `array.0`
- El valor correspondiente se suma en un acumulador (`ecx`)

```
402145: movzx edx, BYTE PTR [rax]
402148: and edx, 0xf
40214b: add ecx, DWORD PTR [rsi + rdx * 4]
```

Este proceso se repite hasta procesar los 6 caracteres.

4. Verificación final Finalmente, se compara la suma total con el valor 56:

```
402157: cmp ecx, 0x38 ; 0x38 = 56
40215a: jne explode_bomb
```

Si el total no coincide, la bomba se activa.

Composición del arreglo de indexación

El arreglo `array.0` está ubicado en la dirección `0x4cde40` y contiene los siguientes valores:

```
array = [2, 13, 7, 14, 5, 10, 6, 15, 1, 12, 3, 4, 11, 8, 16, 9]
# índices: 0 1 2 3 4 5 6 7 8 9 A B C D E F
```

El índice `ord(c) & 0x0F` garantiza que solo se accede a valores entre 0 y 15.

Estrategia de resolución

Para encontrar un string válido, se generaron todas las combinaciones posibles de seis índices entre 0 y 15, y se verificó qué combinaciones sumaban 56 al consultar el arreglo.

Posteriormente, se buscaron caracteres tales que `ord(c) & 0x0F == i` para cada índice `i`, y se ensamblaron strings válidos.

Script auxiliar en Python

```
from itertools import product

array = [2, 13, 7, 14, 5, 10, 6, 15, 1, 12, 3, 4, 11, 8, 16, 9]

for comb in product(range(16), repeat=6):
    if sum(array[i] for i in comb) == 56:
        print(comb)
```

Luego, para cada tupla `comb`, se generaron caracteres compatibles con los índices aplicables.

Ejemplo de string válido

Una de las combinaciones encontradas fue:

001111

Cada uno de estos caracteres cumple que sus 4 bits bajos coinciden con los índices necesarios, y la suma correspondiente es exactamente 56.

Resultado

Al ingresar el string 001111, la función pasó todas las verificaciones y la bomba no se activó, desactivando la fase correctamente.

Fase Secreta – Recorrido Binario con Codificación de Caminos

Activación de la fase

La fase secreta no es parte del flujo principal de ejecución del binario, sino que se invoca desde la función `phase_defused`, una vez que las primeras cuatro fases han sido resueltas exitosamente.

La activación requiere una entrada adicional de tres valores:

```
40261c: lea r8, [rsp+0x10]
402621: lea rsi, [rip+0xc4b1e] ; formato "%d %d %s"
402628: lea rdi, [rip+0xf8c81] ; input_strings[4]
40262f: call __isoc99_sscanf
```

Luego se verifica que el tercer valor ingresado coincida con la cadena "abrete_sesamo":

```
402647: lea rdi, [rsp+0x10]
402653: lea rsi, [rip+0xc4afc] ; → "abrete_sesamo"
402658: call strings_not_equal
```

Si esta comparación retorna cero (son iguales), se invoca la `secret_phase`:

```
402679: call secret_phase
```

Estructura del input

Durante el análisis del binario se comprobó que, de las cinco fases disponibles, **solamente la fase 3 utiliza un sscanf con el formato %s %d**, es decir, string seguido de número. Por lo tanto, se decidió inyectar el string "abrete_sesamo" como **tercer valor** en la línea de entrada utilizada para desactivar la fase 3. Esta línea fue la única que permitió respetar el formato `string número string`.

Después de resolver la fase 4, el programa alcanza la función `phase_defused`, donde se evalúan nuevamente las entradas anteriores. Si el string "abrete_sesamo" fue efectivamente provisto en el lugar esperado, entonces se habilita la ejecución de la fase secreta.

Lógica de `secret_phase`

Esta función espera un nuevo valor numérico, leído desde entrada estándar:

```
4021b2: call read_line
4021c4: call strtol          ; convierte string a número entero
```

El número ingresado debe estar en el rango [1, 1000]:

```
4021cb: sub eax, 1
4021ce: cmp eax, 0x3e8      ; 1000 en decimal
4021d3: ja explode_bomb
```

Luego se llama a la función `fun7`, pasando como argumentos:

- `esi` = `valor_ingresado`
 - `rdi` = `&n1` → raíz de un árbol binario en memoria
-

Función `fun7`: codificación del camino

`fun7` recorre un árbol binario buscando el valor `x`, y construye una codificación del camino basado en decisiones izquierda/derecha:

```
40217b: cmp edx, esi        ; valor actual vs valor buscado
40217d: jg  → izquierda
402184: jne derecha
```

La codificación sigue esta lógica:

- Izquierda: `2 * fun7(left)`
- Derecha: `2 * fun7(right) + 1`
- Hoja encontrada: `return 0`

El valor retornado representa el camino binario desde la raíz hasta el nodo deseado, codificado en forma de número entero.

La fase exige que el resultado de `fun7` sea igual a 2:

```
4021e3: cmp eax, 0x2
4021e6: jne explode_bomb
```

Resolución de la fase

Para desactivar la fase secreta se necesita un valor entre 1 y 1000 tal que, al ser buscado en el árbol con `fun7`, retorne exactamente 2.

Mediante depuración con GDB se inspeccionó la estructura del árbol apuntado por `n1` y se identificó el valor correspondiente al camino `0b10` (izquierda, luego derecha). El número que cumple esta condición es:

22

Resultado

La secuencia completa utilizada fue:

1. Ingresar `abatatar 782 abrete_sesamo` en la entrada correspondiente a `phase_3`, donde:
 - `"abatatar"` y `782` desactivan la fase 3.
 - `"abrete_sesamo"` se almacena como input extra.
2. Completar la fase 4 correctamente.
3. Una vez en `phase_defused`, se detecta el string especial y se activa la fase secreta.
4. Finalmente, se ingresó el número 22, cumpliendo con la condición requerida en `fun7`.

Con esto, la fase secreta fue desactivada exitosamente y la bomba no explotó.